UNIVERSIDADE
LUSÓFONA

Licenciatura em Engenharia Informática

Projeto I

Professor José Braga de Vasconcelos / Ahmed Al-Majmaie a22407520

jose.vasconcelos@ulusofona.pt

Novembro 2026

Software Engineering Report

1. Introduction

1.1. Problem Statement

Atualmente, as organizações contemporâneas, desde equipas de engenharia e operações no terreno até departamentos de suporte, enfrentam um desafio crítico relacionado com a fragmentação do conhecimento interno. A informação técnica, os regulamentos e os procedimentos operacionais encontram-se frequentemente dispersos por dezenas de manuais e documentos PDF extensos. Este ecossistema desorganizado obriga os colaboradores a despenderem horas em pesquisas manuais ineficientes.

Sob pressão operacional, esta latência na recuperação de informação reduz drasticamente a produtividade e aumenta exponencialmente o risco de erro humano, potenciando a aplicação de normas desatualizadas. Em simultâneo, a adoção de soluções genéricas de Inteligência Artificial (como LLMs públicos) introduz riscos inaceitáveis para as empresas, nomeadamente a quebra de confidencialidade (*data leaks*) ao partilhar dados privados com servidores externos, e a ausência de fiabilidade técnica devido à geração de respostas não fundamentadas (*hallucinations*).

1.2. Business Goals

1.2.1. Otimização da Eficiência Operacional

Reduzir drasticamente o tempo de pesquisa (latência de acesso à informação) em documentos técnicos e regulamentares complexos.

1.2.2. Soberania e Segurança de Dados (Zero Data Leakage)

Assegurar que todo o processamento de texto e inferência de IA ocorra num ambiente estritamente local e fechado (*on-premise*), sem qualquer dependência de APIs externas.

1.2.3. Precisão e Rastreabilidade Documental

Eliminar as "alucinações" do modelo de linguagem, garantindo que todas as respostas geradas incluam a citação exata (documento e página) da fonte original, permitindo validação humana imediata.

1.3. Software Project Management

O ciclo de desenvolvimento do sistema adotará uma abordagem iterativa e incremental, dividida estrategicamente entre os dois semestres letivos. O Projeto I concentra-se na especificação rigorosa de requisitos, no desenho da arquitetura de software e na construção de um Produto Mínimo Viável (MVP), enquanto o Projeto II absorve a implementação estrutural, a otimização do Modelo de Linguagem e os testes em ambiente real.

Fase de Desenvolvimento	**Atividades Técnicas e Foco**	**Marco de entrega (*Milestone*)**
Engenharia de Requisitos	Elicitação de requisitos funcionais e não funcionais, identificação de perfis de utilizador, elaboração de casos de uso com critérios de aceitação e matriz de rastreabilidade.	Relatório Preliminar de Engenharia de Requisitos (Prazo: 9 de novembro de 2026).
Arquitetura e Protótipo (MVP)	Desenho dos diagramas de arquitetura, fluxo de dados e interfaces. Programação de um protótipo funcional inicial capaz de demonstrar os fluxos prioritários de pesquisa RAG com dados de teste para validação de risco técnico.	Relatório de Engenharia de Software (Prazo: 16 de dezembro de 2026).
Projeto II	**Implementação**	**2ºSemestre**
Implementação do Pipeline de Dados e IA	Construção do *backend* de produção. Implementação dos algoritmos de extração, limpeza e *chunking* de PDFs reais. Integração robusta via API com o Modelo de Linguagem (LLM) para garantir o bloqueio de "alucinações" em contexto empresarial.	2º Semestre (A definir).
Interface, Segurança e Validação Operacional	Fecho do desenvolvimento da aplicação web e do Painel de Administração. Implementação das políticas de permissões (RBAC) e do sistema de auditoria relacional (*logs*). Condução dos testes de validação com utilizadores operacionais no terreno.	Defesa Final de Projeto II (Fim de 2º Semestre).

1.4. Glossary of Terms

1.4.1. RAG (*Retrieval-Augmented Generation*)

Arquitetura de software que cruza a extração de dados num repositório local com a geração de texto, limitando as respostas da IA estritamente à informação recuperada.

1.4.2. LLM (*Large Language Model*)

Modelo computacional baseado em redes neuronais de aprendizagem profunda, treinado para compreender e gerar linguagem natural.

1.4.3. Local LLM

Modelo de linguagem (ex: Llama 3, Mistral) hospedado e executado numa infraestrutura privada do cliente, garantindo total isolamento dos dados face ao exterior.

1.4.4. Alucinação (*Hallucination*)

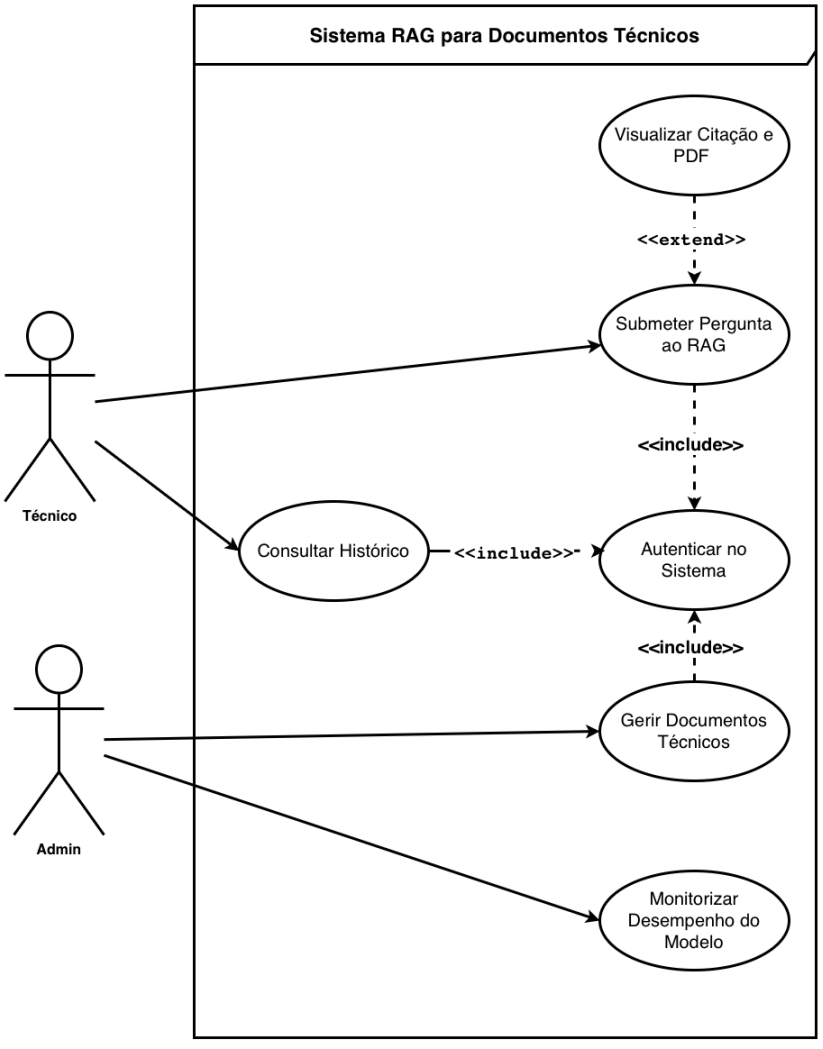
Anomalia intrínseca aos LLMs genéricos, caraterizada pela geração de informações factualmente incorretas ou inexistentes, apresentadas com um elevado grau de confiança.

1.4.5. Vetorização (*Embedding*)

Processo de conversão de blocos de texto em vetores numéricos multidimensionais, permitindo a pesquisa semântica através do cálculo de similaridade.

2. Software Requirements and Analysis

2.1. User Stories and System Requirements



2.2. Functional Requirements

Code	Priority	Description
REQ-01	Alta	O sistema deve permitir o *upload* de ficheiros em formato PDF.
REQ-02	Alta	O sistema deve extrair e vetorizar o texto dos documentos carregados.
REQ-03	Alta	O sistema deve disponibilizar uma interface de *chat* para perguntas em linguagem natural.
REQ-04	Alta	O sistema deve processar as respostas através de um modelo LLM local (ex: Ollama), sem dependência de APIs públicas.
REQ-05	Média	O sistema deve devolver, junto com a resposta, a citação e/ou referência do documento original.
REQ-06	Alta	O sistema deve exigir autenticação (login) aos utilizadores para aceder ao chat e aos documentos.
REQ-07	Média	O sistema deve guardar e permitir a consulta do histórico de interações (perguntas e respostas) de cada utilizador.
REQ-08	Baixa	O sistema deve permitir ao administrador monitorizar métricas básicas de desempenho do modelo local.

2.3. Nonfunctional Requirements

2.3.1. Segurança e Privacidade

O processamento de dados e inferência da IA deve ocorrer em ambiente 100% local, não sendo permitida a partilha de *prompts* ou documentos com servidores externos.

2.3.2. Desempenho

O sistema deve devolver a resposta gerada num tempo aceitável (ex: menos de 10 segundos) após a submissão da pergunta.

2.3.3. Usabilidade

A interface *web* (frontend) deve ser minimalista e intuitiva, semelhante às plataformas de conversação padrão da indústria.

2.3.4. Portabilidade e Compatibilidade

O sistema deve ser independente da plataforma, sendo capaz de correr localmente em diferentes arquiteturas de *hardware* (como processadores Apple Silicon ARM64 e sistemas Windows/x86 tradicionais), recorrendo a ambientes virtuais ou ferramentas de contentorização para facilitar a instalação em qualquer máquina.

2.3.5. Escalabilidade de Dados

A base de dados vetorial deve suportar a indexação de múltiplos manuais complexos em simultâneo (ex: PDFs com mais de 50 páginas), sem que isso degrade a qualidade da pesquisa ou ultrapasse o limite de latência estipulado para o modelo de linguagem.

3. Software Architecture

A arquitetura do sistema baseia-se num padrão Cliente-Servidor moderno, acoplado a um *pipeline* RAG (*Retrieval-Augmented Generation*) executado de forma estritamente local. O sistema divide-se em quatro componentes lógicas principais:

3.1. Frontend (Camada de Apresentação)

Uma aplicação Web responsiva que disponibiliza a interface de conversação (*chat*) para os técnicos e o painel de administração para o carregamento de novos documentos.

3.2. Backend (Camada Lógica e de Orquestração)

O motor do sistema, responsável por gerir as requisições do utilizador. Expõe uma API que recebe as perguntas, processa o carregamento dos PDFs, divide o texto em fragmentos (*chunking*) e orquestra o fluxo de dados entre a base de dados e a Inteligência Artificial.

3.3. Base de Dados Vetorial (*Vector Store*)

Componente de armazenamento otimizada (como ChromaDB ou FAISS) que guarda os *embeddings* (representações numéricas) do texto extraído. É responsável por realizar a pesquisa semântica ultrarrápida, encontrando os parágrafos mais relevantes para responder à pergunta do utilizador.

3.4. Motor de Inferência (Local LLM)

O nó de processamento de Inteligência Artificial, gerido localmente através de ferramentas como o Ollama. Recebe a pergunta do utilizador em conjunto com os parágrafos recuperados da base de dados vetorial, gerando uma resposta precisa sem qualquer ligação à internet (*air gapped*).

4. Software Construction and Implementation

A construção do Produto Mínimo Viável (MVP) seguirá uma abordagem ágil, focada em validar a viabilidade técnica da pesquisa documental conjugada com a inferência local de IA, sem comprometer a latência ou a segurança.

4.1. Stack Tecnológica

A implementação do motor de processamento recorrerá à linguagem Python, aproveitando o seu ecossistema líder na indústria. A orquestração do fluxo RAG será suportada pela *framework* LangChain. Para a interface, adotar-se-ão tecnologias *Web* padrão (HTML, CSS e JavaScript) ou bibliotecas de prototipagem rápida em Python (como o Streamlit), assegurando uma comunicação fluida com o servidor.

4.2. Implementação do Modelo

O processamento de linguagem natural utilizará o *software* Ollama para executar modelos *open-source* quantizados (como o Llama 3). Esta abordagem permite que a IA corra de forma eficiente utilizando os recursos do processador e placa gráfica locais (CPU/GPU), simulando um ambiente de servidor privado empresarial.

4.3. Segurança e Mitigação de Vulnerabilidades

Tratando-se de uma aplicação corporativa, o desenvolvimento do *backend* contemplará defesas contra vetores de ataque comuns em plataformas *web* (como vulnerabilidades de injeção). Adicionalmente, por integrar modelos de linguagem natural, o sistema implementará validações rigorosas na submissão e higienização dos dados de entrada para prevenir ataques de *Prompt Injection*, assegurando que a IA não é manipulada para contornar as restrições de confidencialidade dos documentos indexados.

4.4. Gestão de Projeto e Código

O ciclo de vida do desenvolvimento será alicerçado em práticas sólidas de Engenharia de Software, utilizando o Git para o controlo de versões do código, garantindo a rastreabilidade e a organização durante a fase de programação.

5. Conclusions and future work

A conceção deste sistema RAG local responde diretamente a um desafio crítico no tecido empresarial corporativo: a necessidade premente de aceder de forma ágil, precisa e rastreável ao conhecimento técnico interno, sem comprometer a confidencialidade dos dados da organização. Ao desenhar uma arquitetura que elimina ativamente a dependência de APIs públicas de Inteligência Artificial, o projeto mitiga os riscos de fuga de informação (*data leaks*) e de alucinações cognitivas, estabelecendo uma solução que cumpre os mais elevados padrões de cibersegurança industrial. O Produto Mínimo Viável (MVP) delineado comprova a exequibilidade de democratizar o acesso à informação técnica complexa através de interfaces conversacionais seguras.

Como evolução natural do sistema e perspetivas de trabalho futuro, estão planeadas as seguintes expansões arquitetónicas:

5.1. Controlo de Acessos Baseado em Funções (RBAC)

Implementação de um sistema de permissões hierárquicas, garantindo que as pesquisas do utilizador são restritas ao nível de confidencialidade do seu departamento.

5.2. Reconhecimento Ótico de Caracteres (OCR)

Integração de módulos de visão computacional para permitir a ingestão, extração e vetorização de manuais antigos que se encontrem digitalizados apenas em formato de imagem.

5.3. Escalabilidade de Infraestrutura

Adaptação da arquitetura para implantação em servidores *on-premise* dedicados, com balanceamento de carga e aceleração paralela de GPUs, com vista a suportar dezenas de sessões de utilizadores em simultâneo.